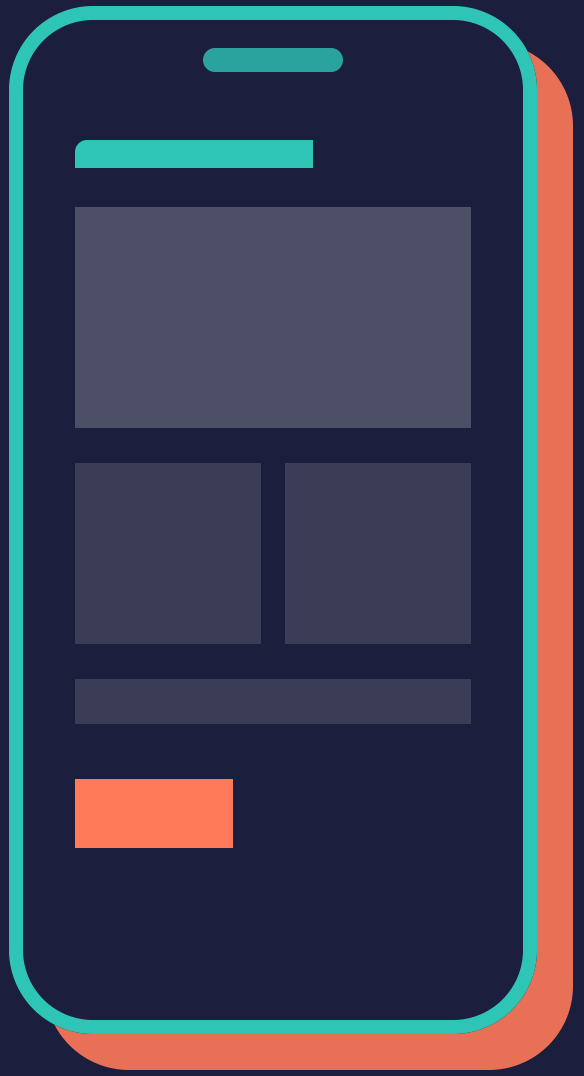
Fondations de React Native

Cours 1 – Architecture, web, composants et responsive

Chris CHEVALIER

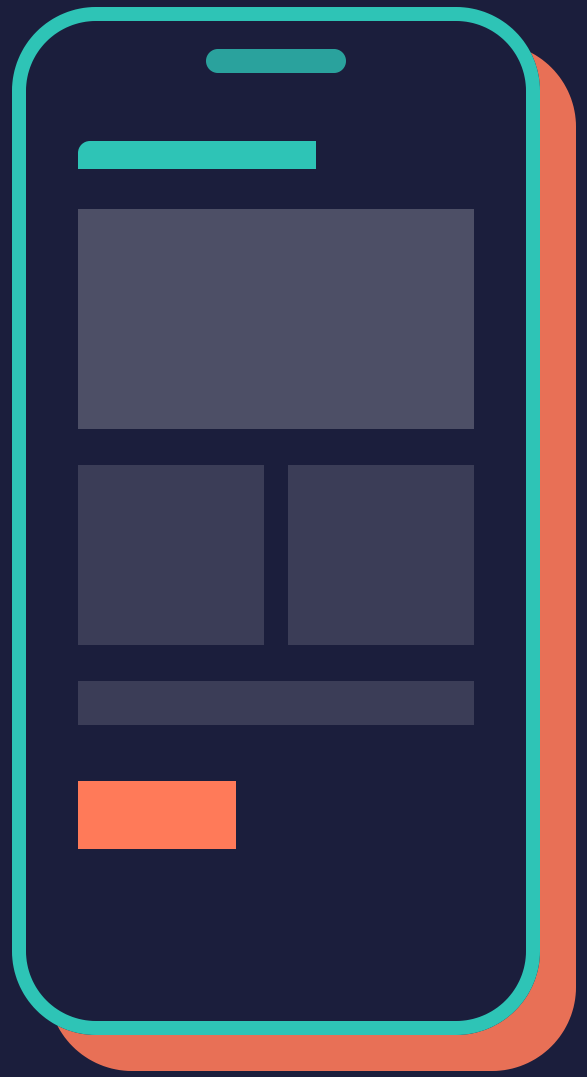


Sommaire

01 De React à React Native

02 Expo et TypeScript

03 Composants, styles et responsive



01

De React à React Native

Ce que vous savez déjà, ce qui change



React et React Native

React

- Une **bibliothèque JavaScript** créée par Meta (ex-Facebook), open source depuis 2013
- Elle construit des **interfaces web** à partir de composants
- Facebook, Instagram, Netflix ou Airbnb l'utilisent sur le web

React Native

- Un **framework** créé par Meta, open source depuis 2015
- Il reprend React pour créer de **vraies applications mobiles** iOS et Android, avec un seul code
- Instagram, Discord, Shopify ou Microsoft Office l'utilisent : voir la vitrine officielle

> React décrit l'interface. Le **moteur de rendu** l'affiche : le DOM du navigateur pour React, les vues natives du téléphone pour React Native.

Le même modèle, une autre cible

	React (web)	React Native
Ce que vous écrivez	Des composants, des props, de l'état, des hooks	
Les composants de base	`<div>`, `<span>`, `<img>`, `<input>`	`View`, `Text`, `Image`, `TextInput`
Les styles	CSS, cascade, sélecteurs	Objets JavaScript, aucune cascade*
Le rendu	Le DOM du navigateur	Des vues natives iOS et Android
La navigation	Des URL	Une pile d'écrans

* Aucune cascade : un style ne s'applique qu'au composant qui le reçoit. Une couleur donnée à un conteneur ne colore pas le texte qu'il contient. Seul un `<Text>` transmet son style à un `<Text>` enfant.

> Votre logique React se réutilise telle quelle. Ce qui change, ce sont les **composants de base** et les **contraintes du mobile**.

Comment JavaScript devient une vue native

1. Votre code

Vos composants TypeScript sont exécutés par **Hermes**, le moteur JavaScript optimisé pour le mobile

2. Le rendu

React calcule l'arbre des composants. Le moteur de rendu **Fabric** le traduit en vues natives

3. L'écran

iOS affiche une `UIView`, Android une `android.view.View`, le navigateur une `<div>`

La nouvelle architecture

JSI (JavaScript Interface) permet des appels **directs** entre JavaScript et le natif. C'est l'architecture par défaut des projets actuels.

React Native for Web

src/components/name-form.tsx

```tsx
import { useState } from "react";
import { StyleSheet, Text, TextInput, View } from "react-native";

export function NameForm() {
  const [name, setName] = useState("");
  return (
    <View>
      <TextInput
        value={name}
        onChangeText={setName}
        placeholder="Votre prénom"
        style={styles.input}
      />
      <Text>{name ? `Bonjour ${name} !` : "…"}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  input: {
```

- React Native for Web réimplémente les composants de React Native avec **React DOM**
- `<View>` devient une `<div>`, `<TextInput>` un `<input>`
- `StyleSheet` génère des **classes CSS** optimisées
- Un seul code pour trois plateformes

Ce que le navigateur reçoit

```html
<div class="css-view-…">
  <input placeholder="Votre prénom"
    class="css-textinput-… r-borderWidth-…">
  <div dir="auto" class="css-text-…">…</div>
</div>
```

React Native for Web (suite)

src/components/name-form.tsx

```
    borderWidth: 1,
    borderRadius: 8,
    padding: 10,
    marginBottom: 8,
  },
});
```

Quiz

Une application React Native peut-elle utiliser `<div>` ?

> Non. `<div>` n'existe que dans le DOM : l'application planterait sur iOS et Android. Utilisez `<View>`, que React Native for Web transforme en `<div>` dans le navigateur.

Une application React Native est-elle une page web dans une application ?

> Non. Sur mobile, ce sont de vraies vues natives. Une page web affichée dans une application s'appelle une **WebView** (c'est l'approche de Cordova ou d'Ionic).

02

Expo et TypeScript

Créer et lancer un projet



Expo : créer et lancer le projet

Terminal

```
npx create-expo-app@latest mon-app
cd mon-app
npm run reset-project
npm start
```

- **Expo** est le framework recommandé par l'équipe de React Native
- `npm run reset-project` retire l'application d'exemple pour partir d'un écran vierge. À sa question, répondez `n` : l'exemple est supprimé au lieu d'être déplacé
- `w` ouvre la version web, `i` le simulateur iOS (macOS et Xcode), `a` l'émulateur Android. Le QR code s'ouvre avec Expo Go sur votre téléphone
- Chaque sauvegarde met l'écran à jour (Fast Refresh)

Indispensable pour le web. Dans `app.json`, remplacez `"output": "server"` par `"output": "single"`. Avec `"server"`, la page est préparée à l'avance, sans connaître la taille de la fenêtre. Un écran qui s'adapte à la largeur plante alors au chargement.

Ce plantage s'appelle une erreur d'**hydratation** : le premier affichage calculé par le navigateur ne correspond pas à la page reçue.

Le téléphone et l'ordinateur doivent être sur le **même réseau**. Si le Wi-Fi de l'école bloque la connexion, lancez `npm start -- --tunnel`.

L'arborescence du projet

mon-app/

```
src/
├── app/              écrans et navigation (Expo Router)
│   ├── _layout.tsx   navigateur racine
│   └── index.tsx     écran de l'URL « / »
├── components/       composants réutilisables
├── hooks/            hooks personnalisés
├── models/           types des données
└── data/             données locales
assets/               images, polices
app.json              configuration Expo
tsconfig.json         configuration TypeScript
```

- Ce n'est pas l'arborescence créée par Expo : après `reset-project`, `src/` ne contient que `app/`. Les autres dossiers suivent les **bonnes pratiques** : un dossier par rôle
- **Chaque fichier de `src/app/` est un écran**. Son chemin donne son URL
- Tout le reste du code vit **en dehors** de `src/app/`
- 💡 L'alias `@/` pointe vers `src/` : `import { STAYS } from "@/data/stays"`

TypeScript : rappels

src/models/stay.ts

```ts
export type StayType = "apartment" | "house" | "cabin";

export type Stay = {
  id: string;
  title: string;
  city: string;
  type: StayType;
  pricePerNight: number;
  available: boolean;
  latitude: number;
  longitude: number;
  imageUrl?: string;
};
```

- `type` décrit la **forme** d'une donnée
- `"apartment" | "house"` : une **union**, seules ces valeurs sont acceptées
- `imageUrl?` : propriété **facultative** (`string | undefined`)
- TypeScript vérifie dans l'éditeur et avec `npx tsc --noEmit` , **avant** l'exécution

03

Composants, styles et responsive

Les composants de base de toute interface



Les composants de base

React Native fournit un petit jeu de **composants de base** (core components). Toute interface s'écrit avec eux.

Composant	Rôle	Équivalent web
`View`	Conteneur et mise en page	`<div>`
`Text`	Tout texte affiché	`<p>`, `<span>`
`Image`	Image locale ou distante	`<img>`
`TextInput`	Saisie de texte	`<input>`
`Pressable`	Zone qui réagit au toucher	`<button>`
`ScrollView`	Contenu qui défile (court)	`overflow: auto`
`FlatList`	Liste longue, affichée au fur et à mesure	Aucun

Tous s'importent depuis `react-native` : `import { View, Text } from "react-native";`

`View` et `Text`

src/components/greeting.tsx

```tsx
import { StyleSheet, Text } from "react-native";

type GreetingProps = { firstName: string; unread?: number };

export function Greeting({ firstName, unread = 0 }: GreetingProps) {
  return (
    <Text style={styles.greeting}>
      Bonjour <Text style={styles.bold}>{firstName}</Text>
      {unread > 0 && ` · ${unread} message(s)`}
    </Text>
  );
}

const styles = StyleSheet.create({
  greeting: { fontSize: 18 },
  bold: { fontWeight: "700" },
});
```

- **Tout texte doit être dans un `<Text>`**. Un texte posé dans une `<View>` fait planter l'application
- Un `<Text>` peut contenir d'autres `<Text>`. Ce sont les seuls à hériter du style de leur parent
- Dans un écran : `<Greeting firstName="Manon" unread={3} />`. Sans `firstName`, TypeScript refuse l'appel
- **Export nommé** pour un composant : `import { Greeting } from "@/components/greeting"`. Seuls les écrans de `src/app/` utilisent `export default`, exigé par Expo Router

Afficher `Greeting` dans un écran

src/app/primitives.tsx

```tsx
import { ScrollView, StyleSheet } from "react-native";

import { Greeting } from "@/components/greeting";

// Page /primitives : les composants de base du cours 1
export default function PrimitivesScreen() {
  return (
    <ScrollView contentContainerStyle={styles.content}>
      <Greeting firstName="Manon" unread={3} />
      <Greeting firstName="Cosmo" />
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  content: { padding: 16, gap: 16 },
});
```

- Chaque fichier de `src/app/` est une page : celui-ci s'ouvre à l'adresse `/primitives`
- On importe le composant, puis on l'utilise comme une balise, avec ses props
- `ScrollView` fait défiler son contenu quand il dépasse l'écran

Image

src/components/image-examples.tsx

```tsx
import { Image, StyleSheet, View } from "react-native";

type ImageExamplesProps = { imageUrl: string };

export function ImageExamples({ imageUrl }: ImageExamplesProps) {
  return (
    <View style={styles.container}>
      {/* Image locale, intégrée à l'application */}
      <Image
        source={require("@/assets/images/icon.png")}
        style={styles.logo}
      />
      {/* Image distante, téléchargée à l'affichage */}
      <Image source={{ uri: imageUrl }} style={styles.photo} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { gap: 16 },
```

- Une image locale passe par `require()` : elle est intégrée à l'application au moment du build
- Une image distante n'a **pas de taille par défaut** : sans largeur ni hauteur, elle reste invisible
- `aspectRatio: 4 / 3` : la hauteur vaut les trois quarts de la largeur, le format d'une photo classique. Avec `width: "100%"`, l'image prend toute la largeur et sa hauteur suit
- Dans le JSX, un commentaire s'écrit `{/* … */}`. Un `//` y deviendrait du texte, ce qui fait planter l'application

Image (suite)

src/components/image-examples.tsx

```
  logo: { width: 64, height: 64 },
  photo: { width: "100%", aspectRatio: 4 / 3, borderRadius: 12 },
});
```

Afficher les images

src/app/primitives.tsx

```tsx
import { ScrollView, StyleSheet } from "react-native";

import { Greeting } from "@/components/greeting";
import { ImageExamples } from "@/components/image-examples";

// Page /primitives : les composants de base du cours 1
export default function PrimitivesScreen() {
  return (
    <ScrollView contentContainerStyle={styles.content}>
      <Greeting firstName="Manon" unread={3} />
      <Greeting firstName="Cosmo" />
      <ImageExamples imageUrl="https://picsum.photos/id/1067/400/300" />
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  content: { padding: 16, gap: 16 },
});
```

- L'URL de l'image distante est une prop : le composant ne dépend d'aucune donnée
- Comparez l'affichage sur le web et sur mobile : le même code, le même rendu

`TextInput` : un champ contrôlé

src/components/name-form.tsx

```tsx
import { useState } from "react";
import { StyleSheet, Text, TextInput, View } from "react-native";

export function NameForm() {
  const [name, setName] = useState("");
  return (
    <View>
      <TextInput
        value={name}
        onChangeText={setName}
        placeholder="Votre prénom"
        style={styles.input}
      />
      <Text>{name ? `Bonjour ${name} !` : "…"}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  input: {
```

- Un champ **contrôlé**, comme en React : sa valeur vient de l'état
- `onChangeText` reçoit directement le texte, sans objet d'événement
- Équivalent web : `<input value onChange>` et `event.target.value`

`TextInput` : un champ contrôlé (suite)

src/components/name-form.tsx

```
21      borderWidth: 1,
22      borderRadius: 8,
23      padding: 10,
24      marginBottom: 8,
25    },
26  });
```

Afficher le formulaire

src/app/primitives.tsx

```tsx
import { ScrollView, StyleSheet } from "react-native";

import { Greeting } from "@/components/greeting";
import { ImageExamples } from "@/components/image-examples";
import { NameForm } from "@/components/name-form";

// Page /primitives : les composants de base du cours 1
export default function PrimitivesScreen() {
  return (
    <ScrollView contentContainerStyle={styles.content}>
      <Greeting firstName="Manon" unread={3} />
      <Greeting firstName="Cosmo" />
      <ImageExamples imageUrl="https://picsum.photos/id/1067/400/300" />
      <NameForm />
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  content: { padding: 16, gap: 16 },
```

- Tapez un prénom : le texte se met à jour à chaque frappe
- Chaque `NameForm` a son propre état : en afficher deux donne deux champs indépendants

Afficher le formulaire (suite)

src/app/primitives.tsx

```
21 });
```

Un retour visuel au toucher

src/components/stay-card.tsx

```tsx
import {
  Image,
  Pressable,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { Stay } from "@/models/stay";

type StayCardProps = {
  stay: Stay;
  isFavorite: boolean;
  onToggleFavorite: (id: string) => void;
};

const formatPrice = (price: number) =>
  new Intl.NumberFormat("fr-FR", {
    style: "currency",
    currency: "EUR",
```

- Sur mobile, il n'y a pas de survol. L'utilisateur doit **voir** que son doigt a été pris en compte
- `style` accepte une fonction qui reçoit `{ pressed }`
- Un tableau de styles applique les suivants par-dessus les premiers. `false` est ignoré

Un retour visuel au toucher (suite)

src/components/stay-card.tsx

```tsx
    maximumFractionDigits: 0,
  }).format(price);

export function StayCard({
  stay,
  isFavorite,
  onToggleFavorite,
}: StayCardProps) {
  return (
    <View style={styles.card}>
      <Image source={{ uri: stay.imageUrl }} style={styles.image} />
      <View style={styles.body}>
        <Text style={styles.title}>{stay.title}</Text>
        <Text style={styles.city}>{stay.city}</Text>
        <Text style={styles.price}>
          {formatPrice(stay.pricePerNight)} / nuit
        </Text>
        {!stay.available && (
          <Text style={styles.unavailable}>Complet</Text>
        )}
```

Un retour visuel au toucher (suite)

src/components/stay-card.tsx

```tsx
        <Pressable
          onPress={() => onToggleFavorite(stay.id)}
          style={({ pressed }) => [
            styles.button,
            pressed && styles.buttonPressed,
          ]}
        >
          <Text style={styles.buttonLabel}>
            {isFavorite ? "★ Favori" : "☆ Ajouter aux favoris"}
          </Text>
        </Pressable>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    flex: 1,
    backgroundColor: "#fff",
```

Un retour visuel au toucher (suite)

src/components/stay-card.tsx

```
    borderRadius: 12,
    overflow: "hidden",
    margin: 8,
  },
  image: { width: "100%", aspectRatio: 4 / 3 },
  body: { padding: 12, gap: 4 },
  title: { fontSize: 18, fontWeight: "700" },
  city: { fontSize: 15, color: "#5A5F80" },
  price: { fontSize: 16, color: "#0B7A70" },
  unavailable: { fontSize: 14, color: "#B42318" },
  button: {
    marginTop: 8,
    paddingVertical: 10,
    borderRadius: 8,
    backgroundColor: "#E2F7F4",
    alignItems: "center",
  },
  buttonPressed: { opacity: 0.6 },
  buttonLabel: { fontSize: 15, fontWeight: "600" },
});
```

StyleSheet et Flexbox

src/components/stay-card.tsx

```tsx
import {
  Image,
  Pressable,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { Stay } from "@/models/stay";

type StayCardProps = {
  stay: Stay;
  isFavorite: boolean;
  onToggleFavorite: (id: string) => void;
};

const formatPrice = (price: number) =>
  new Intl.NumberFormat("fr-FR", {
    style: "currency",
    currency: "EUR",
```

- Des propriétés CSS en **camelCase**, des nombres **sans unité**
- **Aucune cascade** : seul un `<Text>` transmet son style à un `<Text>` enfant
- Flexbox partout, mais `flexDirection` vaut **`column`** par défaut
- `flex: 1` : le composant prend toute la place disponible
- `gap` espace les enfants sans marges

StyleSheet et Flexbox (suite)

src/components/stay-card.tsx

```tsx
    maximumFractionDigits: 0,
  }).format(price);

export function StayCard({
  stay,
  isFavorite,
  onToggleFavorite,
}: StayCardProps) {
  return (
    <View style={styles.card}>
      <Image source={{ uri: stay.imageUrl }} style={styles.image} />
      <View style={styles.body}>
        <Text style={styles.title}>{stay.title}</Text>
        <Text style={styles.city}>{stay.city}</Text>
        <Text style={styles.price}>
          {formatPrice(stay.pricePerNight)} / nuit
        </Text>
        {!stay.available && (
          <Text style={styles.unavailable}>Complet</Text>
        )}
```

StyleSheet et Flexbox (suite)

src/components/stay-card.tsx

```tsx
        <Pressable
          onPress={() => onToggleFavorite(stay.id)}
          style={({ pressed }) => [
            styles.button,
            pressed && styles.buttonPressed,
          ]}
        >
          <Text style={styles.buttonLabel}>
            {isFavorite ? "★ Favori" : "☆ Ajouter aux favoris"}
          </Text>
        </Pressable>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    flex: 1,
    backgroundColor: "#fff",
```

StyleSheet et Flexbox (suite)

src/components/stay-card.tsx

```
    borderRadius: 12,
    overflow: "hidden",
    margin: 8,
  },
  image: { width: "100%", aspectRatio: 4 / 3 },
  body: { padding: 12, gap: 4 },
  title: { fontSize: 18, fontWeight: "700" },
  city: { fontSize: 15, color: "#5A5F80" },
  price: { fontSize: 16, color: "#0B7A70" },
  unavailable: { fontSize: 14, color: "#B42318" },
  button: {
    marginTop: 8,
    paddingVertical: 10,
    borderRadius: 8,
    backgroundColor: "#E2F7F4",
    alignItems: "center",
  },
  buttonPressed: { opacity: 0.6 },
  buttonLabel: { fontSize: 15, fontWeight: "600" },
});
```

Quiz

Je place trois `<Text>` dans une `<View>` sans style. Comment s'affichent-ils ?

> En colonne, l'un sous l'autre : `flexDirection` vaut `column` par défaut.

Je donne `color: "red"` à une `<View>`. Le texte qu'elle contient devient-il rouge ?

> Non. Il n'y a pas de cascade : seul un `<Text>` parent transmet son style à un `<Text>` enfant.

Des données locales typées

src/data/stays.ts

```ts
import { Stay } from "@/models/stay";

export const STAYS: Stay[] = [
  {
    id: "s1",
    title: "Studio lumineux",
    city: "Paris",
    type: "apartment",
    pricePerNight: 95,
    latitude: 48.8566,
    longitude: 2.3522,
    available: true,
    imageUrl: "https://picsum.photos/id/1067/400/300",
  },
  {
    id: "s2",
    title: "Loft avec vue",
    city: "Lyon",
    type: "apartment",
    pricePerNight: 120,
```

- `STAYS` est un tableau de `Stay` : TypeScript vérifie chaque séjour, champ par champ
- Un champ oublié ou une faute de frappe dans `type` est signalé dans l'éditeur
- Les images viennent de picsum.photos, qui tient la charge quand toute la classe ouvre les mêmes images

Des données locales typées (suite)

src/data/stays.ts

```
    latitude: 45.764,
    longitude: 4.8357,
    available: true,
    imageUrl: "https://picsum.photos/id/1031/400/300",
  },
  {
    id: "s3",
    title: "Chalet en bois",
    city: "Annecy",
    type: "cabin",
    pricePerNight: 160,
    latitude: 45.8992,
    longitude: 6.1294,
    available: false,
    imageUrl: "https://picsum.photos/id/1036/400/300",
  },
  {
    id: "s4",
    title: "Maison de pêcheur",
    city: "Biarritz",
```

Des données locales typées (suite)

src/data/stays.ts

```
    type: "house",
    pricePerNight: 140,
    latitude: 43.4832,
    longitude: -1.5586,
    available: true,
    imageUrl: "https://picsum.photos/id/1039/400/300",
  },
  {
    id: "s5",
    title: "Cabane perchée",
    city: "Gérardmer",
    type: "cabin",
    pricePerNight: 110,
    latitude: 48.0731,
    longitude: 6.8784,
    available: true,
    imageUrl: "https://picsum.photos/id/1043/400/300",
  },
  {
    id: "s6",
```

Des données locales typées (suite)

src/data/stays.ts

```
    title: "Villa provençale",
    city: "Aix-en-Provence",
    type: "house",
    pricePerNight: 210,
    latitude: 43.5297,
    longitude: 5.4474,
    available: true,
    imageUrl: "https://picsum.photos/id/1040/400/300",
  },
];
```

Afficher une liste : `FlatList`

src/app/index.tsx

```tsx
import { useState } from "react";
import {
  FlatList,
  StyleSheet,
  Text,
  TextInput,
  View,
} from "react-native";

import { StayCard } from "@/components/stay-card";
import { STAYS } from "@/data/stays";

export default function CatalogScreen() {
  const [query, setQuery] = useState("");
  const [favorites, setFavorites] = useState<string[]>([]);

  const toggleFavorite = (id: string) =>
    setFavorites((current) =>
      current.includes(id)
        ? current.filter((f) => f !== id)
```

- **1.** Les favoris sont un état : `toggleFavorite` ajoute ou retire un identifiant
- **2.** `visibleStays` se calcule à partir de la recherche : ce n'est pas un état
- **3.** `FlatList` ne crée que les cartes **visibles** à l'écran, puis les suivantes au fil du défilement. Une liste de 1 000 séjours reste donc fluide
- `keyExtractor` joue le rôle de la prop `key` de `map()`
- `ScrollView` affiche tout d'un coup : réservez-la aux contenus courts

Afficher une liste : `FlatList` (suite)

src/app/index.tsx

```
        : [...current, id],
    );

  const search = query.trim().toLowerCase();
  const visibleStays = STAYS.filter(
    (stay) =>
      stay.title.toLowerCase().includes(search) ||
      stay.city.toLowerCase().includes(search),
  );

  return (
    <View style={styles.screen}>
      <View style={styles.container}>
        <Text style={styles.heading}>
          Séjours · {favorites.length} favori(s)
        </Text>
        <TextInput
          value={query}
          onChangeText={setQuery}
          placeholder="Rechercher une ville ou un logement"
```

Afficher une liste : `FlatList` (suite)

src/app/index.tsx

```
41          style={styles.search}
42        />
43        <FlatList
44          data={visibleStays}
45          keyExtractor={(stay) => stay.id}
46          renderItem={({ item }) => (
47            <StayCard
48              stay={item}
49              isFavorite={favorites.includes(item.id)}
50              onToggleFavorite={toggleFavorite}
51            />
52          )}
53          ListEmptyComponent={
54            <Text style={styles.empty}>
55              Aucun logement ne correspond.
56            </Text>
57          }
58        />
59      </View>
60    </View>
```

Afficher une liste : `FlatList` (suite)

src/app/index.tsx

```
61    );
62  }
63
64  const styles = StyleSheet.create({
65    screen: { flex: 1, backgroundColor: "#F2F3F9" },
66    container: {
67      flex: 1,
68      width: "100%",
69      maxWidth: 1100,
70      alignSelf: "center",
71      padding: 8,
72    },
73    heading: { fontSize: 24, fontWeight: "800", margin: 8 },
74    search: {
75      margin: 8,
76      padding: 12,
77      borderRadius: 8,
78      borderWidth: 1,
79      borderColor: "#D9DCEA",
80      backgroundColor: "#fff",
```

Afficher une liste : `FlatList` (suite)

src/app/index.tsx

```
    fontSize: 16,
  },
  empty: { margin: 8, fontSize: 16 },
});
```

Pas de media queries : un hook

src/hooks/use-columns.ts

```ts
import { useWindowDimensions } from "react-native";

// Nombre de colonnes selon la largeur de la fenêtre (points de
// rupture)
export function useColumns(): number {
  const { width } = useWindowDimensions();
  if (width >= 1024) return 3;
  if (width >= 600) return 2;
  return 1;
}
```

- `StyleSheet` ne connaît pas les media queries du CSS
- `useWindowDimensions()` renvoie la taille de la fenêtre et **déclenche un nouveau rendu** quand elle change (rotation, redimensionnement)
- Ce n'est pas qu'une question de web : une tablette, un téléphone pliable (Galaxy Z Fold) ou à deux écrans (Surface Duo) change de largeur pendant l'utilisation
- L'ancienne API `Dimensions.get()` ne se met pas à jour toute seule : préférez le hook

Une grille sur le web, une liste sur mobile

src/app/index.tsx

```tsx
import { useState } from "react";
import {
  FlatList,
  StyleSheet,
  Text,
  TextInput,
  View,
} from "react-native";

import { StayCard } from "@/components/stay-card";
import { STAYS } from "@/data/stays";
import { useColumns } from "@/hooks/use-columns";

export default function CatalogScreen() {
  const columns = useColumns();
  const [query, setQuery] = useState("");
  const [favorites, setFavorites] = useState<string[]>([]);

  const toggleFavorite = (id: string) =>
    setFavorites((current) =>
```

- `numColumns` transforme la liste en grille
- `FlatList` refuse de changer de nombre de colonnes à chaud. Changer sa `key` force un nouveau montage
- Sur le conteneur, `maxWidth: 1100` et `alignSelf: "center"` évitent une ligne de 2 500 px sur un grand écran

Une grille sur le web, une liste sur mobile (suite)

src/app/index.tsx

```
      current.includes(id)
        ? current.filter((f) => f !== id)
        : [...current, id],
    );

  const search = query.trim().toLowerCase();
  const visibleStays = STAYS.filter(
    (stay) =>
      stay.title.toLowerCase().includes(search) ||
      stay.city.toLowerCase().includes(search),
  );

  return (
    <View style={styles.screen}>
      <View style={styles.container}>
        <Text style={styles.heading}>
          Séjours · {favorites.length} favori(s)
        </Text>
        <TextInput
          value={query}
```

Une grille sur le web, une liste sur mobile (suite)

src/app/index.tsx

```
41          onChangeText={setQuery}
42          placeholder="Rechercher une ville ou un logement"
43          style={styles.search}
44        />
45        <FlatList
46          key={columns}
47          data={visibleStays}
48          numColumns={columns}
49          keyExtractor={(stay) => stay.id}
50          renderItem={({ item }) => (
51            <StayCard
52              stay={item}
53              isFavorite={favorites.includes(item.id)}
54              onToggleFavorite={toggleFavorite}
55            />
56          )}
57          ListEmptyComponent={
58            <Text style={styles.empty}>
59              Aucun logement ne correspond.
60            </Text>
```

Une grille sur le web, une liste sur mobile (suite)

src/app/index.tsx

```
          }
        />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: "#F2F3F9" },
  container: {
    flex: 1,
    width: "100%",
    maxWidth: 1100,
    alignSelf: "center",
    padding: 8,
  },
  heading: { fontSize: 24, fontWeight: "800", margin: 8 },
  search: {
    margin: 8,
    padding: 12,
```

Une grille sur le web, une liste sur mobile (suite)

src/app/index.tsx

```
81      borderRadius: 8,
82      borderWidth: 1,
83      borderColor: "#D9DCEA",
84      backgroundColor: "#fff",
85      fontSize: 16,
86    },
87    empty: { margin: 8, fontSize: 16 },
88  });
```

À retenir

- React Native affiche de **vraies vues natives**. React Native for Web les traduit en DOM
- **Expo** crée, lance et teste le projet. Chaque fichier de `src/app/` est un écran
- TypeScript décrit vos données et vos props, et vérifie avant l'exécution (`npx tsc --noEmit`)
- Tout texte est dans un `<Text>`. Les styles sont des objets, sans cascade, en colonne par défaut
- Le responsive passe par `useWindowDimensions()` et des points de rupture

> Place à la pratique : le **TP1** occupe toute la fin de la séance. On termine par un **Kahoot de 10 questions**, 15 minutes avant la fin.

Des questions ?

Chris CHEVALIER
chevalier@chris-freelance.com

